SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

TASK A5 — Design the File Storage (4 Marks)

Task / Problem Statement

Store donors and stock in files so that nothing is lost when the computer is switched off, and update one stock record without rewriting the file.

Aim

To choose a suitable file format for each kind of data and to update a record in place with fseek.

A5.1 – Text or Binary? (2 marks)

Donor file – text, one donor per line, written with fprintf:

```
10012,Anitha,O+,24,9876543210,2026-05-01
10023,Karthik,O+,35,9876501234,2026-10-20

fprintf(fp, "%d,%s,%s,%d,%s,%s\n", id, name, group, age, phone, last_donation);
```

- Names and phone numbers have **variable length**, so a text line wastes no space.
- The file can be **read and corrected with any editor**, copied between computers, e-mailed and compared with a diff tool; it does not depend on int size, byte order or struct padding.
- Donors are loaded once into memory, so line-by-line reading speed is acceptable (B3: 100,000 records read in about 21 ms).

Stock file – binary, fixed-size records, written with fwrite:

```
typedef struct { int group_code; int units; } Stock;    /* always 8 bytes */
fwrite(stock, sizeof(Stock), 8, fp);      /* record i starts at byte i * sizeof(Stock) */
```

- Every record has the **same size**, so record *i* is at byte offset i × sizeof(Stock) and can be reached directly with fseek (random access).
- Stock changes after **every donation and issue**; a binary file lets one record be changed in O(1) without touching the others.
- No text conversion is needed (fast, exact integers).

A5.2 – Update in Place (2 marks)

```
ALGORITHM UpdateUnits(fp, index, units)
  INPUT  : fp = binary stock file opened "r+b", index = record number, units = new value
  OUTPUT : 1 on success, 0 on failure
BEGIN
  IF fseek(fp, index * sizeof(Stock), SEEK_SET) <> 0 THEN RETURN 0     // go to the record
  IF fread(&s, sizeof(Stock), 1, fp) <> 1 THEN RETURN 0              // read it (fails past end of file)
  s.units <- units                                    // change it in memory
  IF fseek(fp, -sizeof(Stock), SEEK_CUR) <> 0 THEN RETURN 0          // step back one record
  IF fwrite(&s, sizeof(Stock), 1, fp) <> 1 THEN RETURN 0             // write it back
  RETURN 1
END
```

The step back is needed because fread moved the file position to the start of the next record; without it the write would damage the following record. A seek is required whenever a program switches between reading and writing the same file.

Why this is difficult for the text file. Lines have **different lengths**, so the position of line *k* cannot be calculated – the file must be read from the start to find it. If the new text has a different length (e.g. 9 → 10), every byte after it would have to move, which a file cannot do. The only safe method is to rewrite the whole file (usually to a temporary file that then replaces the original).

Alternative Concept and Complexity Analysis – text stock file rewritten after every update

	Binary file, update in place	Text file, rewrite whole file
Time per update (n records)	**O(1)**: one seek, one read, one write	**O(n)**: read and parse n lines, write n lines

	Binary file, update in place	Text file, rewrite whole file
Measured (B3.3, 100,000 donor records)	0.0022 ms	8.14 ms (3,696 × slower)
File size	Exactly n × 8 bytes (8 groups → 64 bytes)	About 6–8 bytes per line (O+,40\n), variable – slightly smaller or similar for small numbers
Crash safety	Interrupted write damages one record only	Interrupted rewrite can truncate the file; needs temp file + rename
Human readable	No	Yes

For only 8 stock records either is fast, but the stock file is updated constantly and the design must stay efficient if the district grows (more groups, more hospitals), so the in-place binary design is chosen.

Result

Text format for donors and binary fixed-size format for stock were justified, and an in-place update algorithm was given with its O(1) versus O(n) comparison.